

A Pannon Egyetem gyakorlatorientált infrastrukturális- és készségfejlesztési reformja

RRF-2.1.2.-21-2022-00007

NUMERICAL ANALYSIS

Ferenc Hartung, University of Pannonia



Pannon Egyetem
University of Pannonia

Numerical Analysis

1 Introduction

Ferenc Hartung

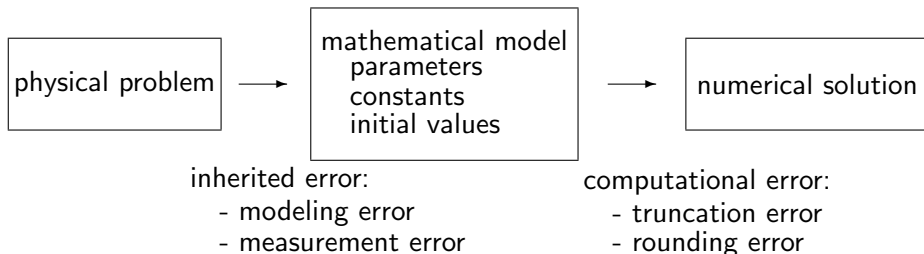
University of Pannonia
Department of Mathematics
Veszprém, Hungary



2025

1.1 The Main Objective and Notions of Numerical Analysis

Schematic steps of the scientific computation of physical processes:



The **main objective** of numerical analysis is to give exact or approximate solutions of a mathematical problem using arithmetic operations (addition, subtraction, multiplication and division).

We get a **truncation error** when we replace the exact value of a mathematical expression with an approximate formula.

Example

The goal is to compute the function value of $f(x) = \sin x$ at x .

Consider the Taylor-polynomial around 0 of degree 5:

$$T_5(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!}.$$

The Taylor's theorem yields

$$f(x) = T_5(x) + \frac{f^{(6)}(\xi)}{6!}x^6.$$

The truncation error is

$$\frac{f^{(6)}(\xi)}{6!}x^6 = -\frac{\sin \xi}{6!}x^6.$$

The **rounding error** appears since real numbers can be stored in computers with finite digits accuracy.

The next notion we discuss related to a numerical algorithm is the **stability**. This notion is used in two meanings in numerics. We can talk about the **stability of a mathematical model** or about the **stability of a numerical method**.

Example

Consider the linear system

$$7x + 505y = 940$$

$$6x + 433y = 806.$$

Its exact solution is $x = -10$ and $y = 2$.

Next consider

$$7x + 505y = 940$$

$$6.01x + 433y = 806.$$

Its solution is $x = 2.4691$ and $y = 1.8272$. We observe that 0.17% change in the size of the coefficient results in 124.7% and 8.6% change in the solutions.

We say that a mathematical problem is **correct** or **stable** or **well-conditioned**, if a “small” change in the parameters of the problem results only in “small” change in the solution of the problem. In the opposite case we say that the problem is **incorrect** or **ill-conditioned** or it is an **unstable problem**. The system in the previous example is incorrect.

We say that a numerical algorithm is **stable with respect to rounding errors** if the rounding errors do not influence the result of the computation significantly. If the computed result is significantly different from the true value, then we say that the **algorithm is unstable**.

Example

Consider the following three recursive sequences:

$$x_n = \frac{1}{3}x_{n-1}, \quad x_0 = 1,$$

$$y_n = 2y_{n-1} - \frac{5}{9}y_{n-2}, \quad y_0 = 1, \quad y_1 = \frac{1}{3},$$

$$z_n = \frac{13}{3}z_{n-1} - \frac{4}{3}z_{n-2}, \quad z_0 = 1, \quad z_1 = \frac{1}{3}.$$

It is easy to show that $x_n = y_n = z_n = \frac{1}{3^n}$, i.e., the three sequences are algebraically equivalent. Next we use *singe precision* floating point arithmetic:

n	x_n	y_n	$ y_n - 1/3^n $	z_n	$ z_n - 1/3^n $
2	0.111111	0.111111	2.2352e-08	0.111111	4.4703e-08
3	0.037037	0.037037	4.0978e-08	0.037037	1.8254e-07
4	0.012346	0.012346	6.9849e-08	0.012346	7.3109e-07
5	0.004115	0.004115	1.1688e-07	0.004118	2.9248e-06
6	0.001372	0.001372	1.9465e-07	0.001383	1.1699e-05
7	0.000457	0.000458	3.2442e-07	0.000504	4.6795e-05
8	0.000152	0.000153	5.4071e-07	0.000340	1.8718e-04
9	0.000051	0.000052	9.0117e-07	0.000800	7.4872e-04
10	0.000017	0.000018	1.5019e-06	0.003012	2.9949e-03
11	0.000006	0.000008	2.5032e-06	0.011985	1.1980e-02
12	0.000002	0.000006	4.1721e-06	0.047920	4.7918e-02
13	0.000001	0.000008	6.9535e-06	0.191674	1.9167e-01
14	0.000000	0.000012	1.1589e-05	0.766693	7.6669e-01
15	0.000000	0.000019	1.9315e-05	3.066773	3.0668e+00
16	0.000000	0.000032	3.2192e-05	12.267091	1.2267e+01
17	0.000000	0.000054	5.3653e-05	49.068363	4.9068e+01
18	0.000000	0.000089	8.9422e-05	196.273453	1.9627e+02

Example (cont.):

Using a *double precision* floating point arithmetic the error of the 18th terms:

$$|y_{18} - 1/3^{18}| = -2.5104\text{e-}13 \quad \text{and} \quad |z_{18} - 1/3^{18}| = 2.3804\text{e-}07.$$

In case of an algorithm which terminates in a finite number of steps we are usually interested in the **time complexity** or the **cost** of an algorithm. By this we mean the number of steps, or more precisely, the **number of arithmetic operations** needed to perform the algorithm.

Example

Evaluate numerically the polynomial $p(x) = 6x^4 - 3x^3 + 5x^2 + 8x - 10$ at the point x .

It contains 4 additions/subtractions, 4 multiplications and 3 exponentials. The exponentials mean $3+2+1=6$ number of multiplications, i.e., altogether 10 multiplications are needed to apply the formula of p . But we can rewrite p as follows:

$$p(x) = 6x^4 - 3x^3 + 5x^2 + 8x - 10 = (((6x - 3)x + 5)x + 8)x - 10.$$

This form of the polynomial requires only 4 additions/subtractions and 4 multiplications.

The previous method can be extended to polynomials of degree n :

$$a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0 = (((\dots ((a_n x + a_{n-1})x + a_{n-2})x + \dots)x + a_1)x + a_0$$

This formula requires only n additions/subtractions and n multiplications. This way of organizing a polynomial evaluation is called **Horner's method**.

Algorithm: Horner's method

INPUT: n - degree of the polynomial

$a_n, a_{n-1}, \dots, a_0$ - coefficients of the polynomial

x - argument

OUTPUT: p - function value of the polynomial at the argument x

$p \leftarrow a_n$

for $i = n - 1, \dots, 0$ **do**

$p \leftarrow a_i + px$

end do

output(p)

It is also important to know the **space complexity** of an algorithm, which is the amount of the memory storage needed in the worst case at any point in the algorithm. When we work with an algorithm to solve a linear system with a 10×10 coefficient matrix, the storage cannot be a problem. But the same with a 10000×10000 dimensional matrix can be problematic. In case of algorithms working with a big amount of data, we prefer a method which requires less amount of memory space.

1.2 Computer Representation of Reals

We recall that the real number in base b number system

$$x = (x_{m-1}x_{m-2} \cdots x_0.x_{-1}x_{-2} \cdots)_b, \quad x_i \in \{0, 1, \dots, b-1\}$$

has the numerical value

$$x = x_{m-1}b^{m-1} + x_{m-2}b^{m-2} + \cdots + x_1b + x_0 + \frac{x_{-1}}{b} + \frac{x_{-2}}{b^2} + \cdots = \sum_{i=-\infty}^{m-1} x_i b^i.$$

Consider the real number 126.42. Different books define the normal form of this number as $1.2642 \cdot 10^2$ or $0.12642 \cdot 10^3$. In this lecture notes we use the first form as the normal form.

Therefore, the **normal form** of a real number $x \neq 0$ in a base b number system is

$$x = \pm m \cdot b^k, \quad \text{where } 1 \leq m < b.$$

The number m is called the **mantissa**, and k the **exponent** of the number.

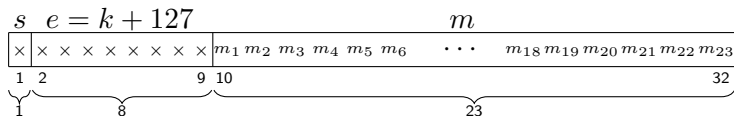
In order to represent a real number, or other words, a **floating point** number we write it in a normal form in a base b number system, and we would like to store its signed mantissa and exponent. The computers use different number of bits to store these numbers. Here we present an IEEE specification (IEEE Binary Floating Point Arithmetic Standard, 754-1985) to represent floating point numbers on 32 bits (the so-called **single precision**), and on 64 bits (the **double precision**) using the binary number system.

Representation on 32 bits:

For $x \neq 0$ consider the binary normal form

$$x = (-1)^s m \cdot 2^k, \quad \text{where } s \in \{0, 1\} \quad \text{and} \quad m = (1.m_1 m_2 m_3 \dots)_2.$$

The value s is stored in the 1st bit. Instead of the exponent k , we store its shifted value, the nonnegative integer $e = k + 127$ on bits 2–9. In our definition of the binary normal form, a nonzero x has a mantissa of the form $m = (1.m_1 m_2 \dots)_2$, i.e., it always starts with 1, which we do not store, we store the fractional digits of the mantissa rounded to 23 bits. The 23 bits are stored on the 10–32 bits of the storage.



This IEEE specification defines the representation of the number 0, and introduces two special symbols: `Inf` (to store infinity as a possible value) and `NaN` (not-a-number):

number	s	e (2–9 bits)	mantissa bits (10–32 bits)
+0	0	00000000	every mantissa bit=0
−0	1	00000000	every mantissa bit=0
+Inf	0	11111111	at least one mantissa bit=0
−Inf	1	11111111	at least one mantissa bit=0
+NaN	0	11111111	every mantissa bit=1
−NaN	1	11111111	every mantissa bit=1

The symbol `Inf` can be used in programs as a result of a mathematical operation with value ∞ , and the symbol `NaN` can be a result of a mathematical operation which is undefined (e.g., a division by 0 or a root of a negative number in real numbers).

The definition yields that the exponent $e = (11111111)_2 = 255$ is used exclusively for the special symbols Inf and NaN.

For finite reals the possible values are

$$0 \leq e = k + 127 \leq 254,$$

hence the possible values of the exponent k are

$$-127 \leq k \leq 127.$$

Therefore, the smallest positive representable number corresponds to

$$k = -127 \quad \text{and} \quad m = (1.00 \dots 01)_2.$$

Hence its value is

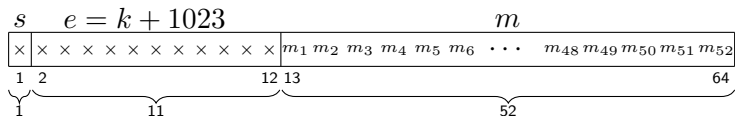
$$x_{\min} = (1.00 \dots 01)_2 \cdot 2^{-127} = (1 + 2^{-23})2^{-127} \approx 10^{-38}.$$

The largest real can be stored is

$$x_{\max} = (1.11 \dots 1)_2 \cdot 2^{127} = (2 - 2^{-23})2^{127} \approx 10^{38}.$$

The **representation on 64 bits** is similar:

The shifted exponent $e = k + 1023$ is stored on bits 2–12, the fractional part of the mantissa is stored on bits 13–64.

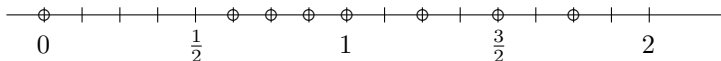


Then the positive real numbers can be stored in the computer are, approximately, in between 10^{-308} and 10^{308} .

Example

Suppose we would like to store reals on 4 bits using a binary normal form. For example, we use the 1st bit as the sign bit, the shifted exponent $e = k + 1$ is stored on the 2nd bit, and the fractional part of the mantissa is stored on bits 3–4. (The symbols Inf and NaN are not defined now.)

s	e	m	x
0	0	00	0
0	0	01	$(1.01)_2 \cdot 2^{-1} = (1 + \frac{1}{4})\frac{1}{2} = \frac{5}{8}$
0	0	10	$(1.10)_2 \cdot 2^{-1} = (1 + \frac{1}{2})\frac{1}{2} = \frac{3}{4} = \frac{6}{8}$
0	0	11	$(1.11)_2 \cdot 2^{-1} = (1 + \frac{1}{2} + \frac{1}{4})\frac{1}{2} = \frac{7}{8}$
0	1	00	$(1.00)_2 \cdot 2^0 = 1 = \frac{8}{8}$
0	1	01	$(1.01)_2 \cdot 2^0 = 1 + \frac{1}{4} = \frac{10}{8}$
0	1	10	$(1.10)_2 \cdot 2^0 = 1 + \frac{1}{2} = \frac{12}{8}$
0	1	11	$(1.11)_2 \cdot 2^0 = 1 + \frac{1}{2} + \frac{1}{4} = \frac{7}{4} = \frac{14}{8}$



The reals which can be stored without an error in a certain floating point representation are called **machine numbers**. The machine number which is stored in a computer instead of the real number x is denoted by $\text{fl}(x)$.

$$\text{fl}(x) = \begin{cases} 0, & |x| < x_{\min}, \\ \text{Inf}, & x > x_{\max}, \\ -\text{Inf}, & x < -x_{\max}. \end{cases}$$

In the first case we talk about arithmetic **underflow**, and in the second and third cases, about arithmetic **overflow**.

For the definition of $\text{fl}(x)$ in the intermediate cases there are basically two methods: In the first case we take the binary normal form of x , consider its mantissa $m = (1.m_1m_2m_3\dots)_2$, and we consider as many first several mantissa fractional bits as it is possible to store in the particular representation, store them, and omit the rest of the mantissa bits. This method is called **chopping** of the mantissa.

Using **rounding**, the mantissa of $\text{fl}(x)$ is defined so that $\text{fl}(x)$ be the nearest machine number to x . The IEEE specification for single precision representation we defined above uses the following rule: Let the normal form of $x > 0$ be

$$x = m2^k, \quad \text{where } m = (1.m_1m_2\dots m_{23}m_{24}\dots)_2.$$

Let

$$x' = (1.m_1m_2\dots m_{23})_2 2^k \quad \text{and} \quad x'' = ((1.m_1m_2\dots m_{23})_2 + 2^{-23}) 2^k.$$

Then x' and x'' are consecutive machine numbers,

$$x' \leq x \leq x'' \quad \text{and} \quad x'' - x' = 2^{k-23}.$$

The specification defines

$$\text{fl}(x) = \begin{cases} x', & \text{if } |x - x'| < \frac{1}{2}|x'' - x'|, \\ x'', & \text{if } |x - x''| < \frac{1}{2}|x'' - x'|, \\ x', & \text{if } |x - x'| = \frac{1}{2}|x'' - x'| \text{ and } m_{23} = 0, \\ x'', & \text{if } |x - x'| = \frac{1}{2}|x'' - x'| \text{ and } m_{23} = 1. \end{cases}$$

Using the rounding, the error is

$$|x - \text{fl}(x)| \leq \frac{1}{2}|x'' - x'| = \frac{1}{2}2^{-23}2^k.$$

If we compare it to the exact value we get

$$\frac{|x - \text{fl}(x)|}{|x|} \leq \frac{|x - \text{fl}(x)|}{(1.m_1m_2\dots)_2 \cdot 2^k} \leq \frac{1}{2}2^{-23}.$$

We can see that the first machine number which is larger than 1 is $1 + 2^{-23}$ in the single precision floating point arithmetic. Let ε_m denote the difference of the first machine number right to 1 and the number 1. This number is called **machine epsilon**. Therefore, ε_m is the smallest power of 2 (in a binary storage system) for which the computer evaluates the inequality $1 + \varepsilon_m > 1$ to be true.

Theorem

Let $0 < \text{fl}(x) < \text{Inf}$, and suppose the floating point representation uses rounding. Then

$$\frac{|x - \text{fl}(x)|}{|x|} \leq \frac{1}{2}\varepsilon_m.$$

Let x be a real number, and consider $\tilde{x}$ as its approximation. Then the **absolute error** or just simply the **error** of the approximation is the number

$$|x - \tilde{x}|.$$

Example

Let $x = 10000$ and $\tilde{x} = 10000.1$. Then the error is $|x - \tilde{x}| = 0.1$.
Now let $y = 1$ and $\tilde{y} = 1.1$. Then the error is $|y - \tilde{y}| = 0.1$.

The **relative error** is defined by

$$\frac{|x - \tilde{x}|}{|x|} \quad (x \neq 0).$$

Example

Let $x = 10000$ and $\tilde{x} = 10000.1$. Then the relative error is $|x - \tilde{x}|/x = 10^{-5}$.
Now let $y = 1$ and $\tilde{y} = 1.1$. Then the error is $|y - \tilde{y}| = 10^{-1}$.

4-digit arithmetic: a floating point arithmetic using a decimal number system with 4 stored mantissa digits (and suppose we can store any exponent). This means that, in every steps of a calculation, the result is rounded to the first 4 **significant digits**.

Example

Using a 4-digit arithmetic we get

$$1.043 + 32.25 = 33.29,$$

note the exact sum is 33.293.

Similarly,

$$1.043 \cdot 32.25 = 33.64$$

(after rounding the exact value 33.63675).

But

$$1.043 + 20340 = 20340,$$

since we rounded the exact value 20341.043 to 4 significant digits.

1.3 Error Analysis

Let x and y be positive real numbers, and consider the numbers $\tilde{x}$ and $\tilde{y}$ as an approximation of x and y . Let

$$|x - \tilde{x}| \leq \Delta_x \quad \text{and} \quad |y - \tilde{y}| \leq \Delta_y$$

be error bounds of the approximation. The relative error bounds are denoted by

$$\delta_x := \frac{\Delta_x}{x} \quad \text{and} \quad \delta_y := \frac{\Delta_y}{y},$$

respectively.

We examine the following question: We would like to perform an arithmetic operation (addition, subtraction, multiplication or division) on the real numbers x and y , but instead of it, we perform the operation on the numbers $\tilde{x}$ and $\tilde{y}$ (suppose without an error).

Consider first the addition. We are looking for error bounds Δ_{x+y} and δ_{x+y} such that

$$|x + y - (\tilde{x} + \tilde{y})| \leq \Delta_{x+y} \quad \text{and} \quad \frac{|x + y - (\tilde{x} + \tilde{y})|}{x + y} \leq \delta_{x+y}.$$

Theorem

The numbers

$$\Delta_{x+y} := \Delta_x + \Delta_y \quad \text{and} \quad \delta_{x+y} := \max\{\delta_x, \delta_y\}$$

are absolute and relative error bounds of the addition, respectively.

Proof

Using the triangle inequality and the definitions of Δ_x and Δ_y , we get

$$|x + y - (\tilde{x} + \tilde{y})| \leq |x - \tilde{x}| + |y - \tilde{y}| \leq \Delta_x + \Delta_y.$$

This means that $\Delta_x + \Delta_y$ is an upper bound of the error of the addition. Using the above relation, we obtain

$$\begin{aligned} \frac{|x + y - (\tilde{x} + \tilde{y})|}{x + y} &\leq \frac{\Delta_x + \Delta_y}{x + y} = \frac{\Delta_x}{x + y} + \frac{\Delta_y}{x + y} \\ &= \frac{x}{x + y} \frac{\Delta_x}{x} + \frac{y}{x + y} \frac{\Delta_y}{y} = \frac{x}{x + y} \delta_x + \frac{y}{x + y} \delta_y \\ &\leq \max\{\delta_x, \delta_y\}. \end{aligned}$$

Therefore, $\max\{\delta_x, \delta_y\}$ is a relative error bound of the addition.

Certainly, the theorem gives the worst case estimate. In practice the errors can balance each other.

Example

Let $x = 1$, $y = 2$, $\tilde{x} = 1.1$ and $\tilde{y} = 1.8$. Then $x + y = 3$ and $\tilde{x} + \tilde{y} = 2.9$. Therefore the error of the sum

$$|x + y - (\tilde{x} + \tilde{y})| = 0.1,$$

but

$$\Delta_x + \Delta_y = 0.1 + 0.2 = 0.3.$$

Theorem

Let $x > y > 0$. The numbers

$$\Delta_{x-y} := \Delta_x + \Delta_y \quad \text{and} \quad \delta_{x-y} := \frac{x}{x-y}\delta_x + \frac{y}{x-y}\delta_y$$

are absolute and relative error bounds of the subtraction.

Proof

The inequalities

$$|x - y - (\tilde{x} - \tilde{y})| = |x - \tilde{x} - (y - \tilde{y})| \leq |x - \tilde{x}| + |y - \tilde{y}| \leq \Delta_x + \Delta_y$$

imply the first statement. Consider

$$\begin{aligned} \frac{|x - y - (\tilde{x} - \tilde{y})|}{x - y} &\leq \frac{\Delta_x + \Delta_y}{x - y} \\ &= \frac{x}{x - y} \frac{\Delta_x}{x} + \frac{y}{x - y} \frac{\Delta_y}{y} \\ &= \frac{x}{x - y} \delta_x + \frac{y}{x - y} \delta_y, \end{aligned}$$

which gives the second statement.

We can observe that if we **subtract two nearly equal numbers**, then the relative error can be magnified compared to the relative error of the terms. In other words, the number of exact digits can be significantly less than in the original numbers. This phenomenon is called **loss of significance**.

Example

Let $x = 12.47531$, $\tilde{x} = 12.47534$, $y = 12.47326$ and $\tilde{y} = 12.47325$. Then

$$\delta_x \approx 2.4 \cdot 10^{-6} \quad \text{and} \quad \delta_y \approx 8 \cdot 10^{-7}.$$

On the other hand,

$$x - y = 0.00205, \quad \tilde{x} - \tilde{y} = 0.00209,$$

and so

$$\delta_{x-y} \approx 0.0195.$$

Theorem

Let $x, y > 0$. The numbers

$\Delta_{x \cdot y} := x\Delta_y + y\Delta_x + \Delta_x\Delta_y$, and $\delta_{x \cdot y} := \delta_x + \delta_y + \delta_x\delta_y$
are relative and absolute error bounds of the multiplication, respectively.

Proof

The triangle-inequality yields

$$\begin{aligned}
 |xy - \tilde{x}\tilde{y}| &= |xy - x\tilde{y} + x\tilde{y} - \tilde{x}\tilde{y}| \\
 &\leq x|y - \tilde{y}| + |\tilde{y}||x - \tilde{x}| \\
 &\leq x\Delta_y + |\tilde{y}|\Delta_x \\
 &= x\Delta_y + |y + \tilde{y} - y|\Delta_x \\
 &\leq x\Delta_y + y\Delta_x + \Delta_x\Delta_y.
 \end{aligned}$$

Using the first part, we get

$$\frac{|xy - \tilde{x}\tilde{y}|}{xy} \leq \frac{x\Delta_y + y\Delta_x + \Delta_x\Delta_y}{xy} = \delta_x + \delta_y + \delta_x\delta_y,$$

which implies the second statement.

Since, in general, Δ_x and Δ_y are much smaller than x and y , and so $\Delta_x \Delta_y$ is much smaller than $x \Delta_y$ and $y \Delta_x$, we have that

$$\Delta_{x \cdot y} \approx x \Delta_y + y \Delta_x$$

. Similarly,

$$\delta_{x \cdot y} \approx \delta_x + \delta_y$$

. Both results mean that the errors do not propagate rapidly in multiplication.

Theorem

Suppose $x, y > 0$ and $\delta_y < 1$. Then the numbers

$$\Delta_{x/y} := \frac{x\Delta_y + y\Delta_x}{y(y - \Delta_y)} \quad \text{and} \quad \delta_{x/y} := \frac{\delta_x + \delta_y}{1 - \delta_y}$$

are absolute and relative error bounds of the division, respectively.

Proof

Elementary manipulations give

$$\left| \frac{x}{y} - \frac{\tilde{x}}{\tilde{y}} \right| = \frac{|x\tilde{y} - xy + xy - \tilde{x}y|}{y|\tilde{y}|} \leq \frac{x\Delta_y + y\Delta_x}{y|\tilde{y}|} = \frac{x\Delta_y + y\Delta_x}{y|y - (y - \tilde{y})|}.$$

Assumption $\delta_y < 1$ implies $|y - \tilde{y}| \leq \Delta_y < y$, hence

$|y - (y - \tilde{y})| \geq y - |y - \tilde{y}| \geq y - \Delta_y > 0$ proves the first statement.

For the second part, consider

$$\frac{\left| \frac{x}{y} - \frac{\tilde{x}}{\tilde{y}} \right|}{\frac{x}{y}} = \frac{|x(\tilde{y} - y) - y(\tilde{x} - x)|}{x|\tilde{y}|} = \frac{\left| \frac{\tilde{y} - y}{y} - \frac{\tilde{x} - x}{x} \right|}{\left| 1 - \frac{y - \tilde{y}}{y} \right|} \leq \frac{\delta_x + \delta_y}{1 - \delta_y}.$$

If δ_y is small, then

$$\delta_{x/y} \approx \delta_x + \delta_y.$$

Similarly, if Δ_y is much smaller than y , then

$$\Delta_{x/y} \approx \frac{1}{y} \Delta_x + \frac{x}{y^2} \Delta_y.$$

If y is much smaller than x , or if y is close to 0, then Δ_y or Δ_x can be significantly magnified, so the absolute error can be much larger than the absolute error of the terms.

Example

Let $x = 42.721531$, $\tilde{x} = 42.721534$, $y = 0.00324721$ and $\tilde{y} = 0.00324732$.
Then

$$\Delta_x = 3 \cdot 10^{-6} \quad \text{and} \quad \Delta_y = 1.1 \cdot 10^{-7}.$$

On the other hand,

$$\frac{x}{y} \approx 13156.38071, \quad \frac{\tilde{x}}{\tilde{y}} \approx 13155.93597,$$

and so

$$\Delta_{x/y} \approx 0.44474.$$

So the error in the result of the division is much bigger than the errors in x and y .

1.4 The Consequences of the Floating Point Arithmetic

Example

Solve the equation

$$x^2 - 83.5x + 1.5 = 0$$

using 4-digit arithmetic in the computations. Using the quadratic formula and the 4-digit arithmetic we get

$$\tilde{x} = \frac{83.5 \pm \sqrt{83.5^2 - 4 \cdot 1.5}}{2} = \frac{83.5 \pm \sqrt{6972 - 6.000}}{2} = \frac{83.5 \pm 83.46}{2},$$

hence

$$\tilde{x}_1 = \frac{167.0}{2} = 83.50 \quad \text{and} \quad \tilde{x}_2 = \frac{0.040}{2} = 0.020.$$

We can check that the exact solutions are

$$x_1 = 83.482032 \quad \text{and} \quad x_2 = 0.0179679$$

(with several digits precision), hence the relative error bounds for each roots:

$$\delta_1 = 0.0002152 \quad \text{and} \quad \delta_2 = 0.113096.$$

The relative error of x_2 is much bigger than that of x_1 .

Consider the second root of $ax^2 + bx + c = 0$:

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}. \quad (1)$$

When b is negative and $4ac$ is much smaller than b^2 , then we subtract to nearly equal numbers, and we observe the loss of significance. (This was the case in the previous example) To avoid this problem, consider

$$x_2 = \frac{b^2 - (b^2 - 4ac)}{2a(-b + \sqrt{b^2 - 4ac})} = \frac{2c}{-b + \sqrt{b^2 - 4ac}}. \quad (2)$$

This formula is algebraically equivalent to formula (1). But the difference is that here we do not subtract two close numbers (in the denominator we add two positive numbers). If b is positive, then for the first root we get

$$x_1 = \frac{2c}{-b - \sqrt{b^2 - 4ac}}. \quad (3)$$

Example

Compute the second root of the equation of the previous example using 4-digit arithmetic and formula (2).

$$\tilde{x}_2 = \frac{2 \cdot 1.5}{83.5 + \sqrt{83.5^2 - 4 \cdot 1.5}} = \frac{3}{83.5 + 83.46} = \frac{3}{167.0} = 0.01796.$$

The relative error of this root is now $\delta_2 = 0.00044$.

Example

Suppose we need to evaluate the expression

$$\cos^2 x - \sin^2 x.$$

If $x = \frac{\pi}{4}$, then the exact value of this expression is 0, hence if x is close to $\frac{\pi}{4}$, then in the expression we need to subtract two nearly equal numbers, so we can face loss of significance. We can use

$$\cos^2 x - \sin^2 x = \cos 2x.$$

In the previous examples we used algebraic manipulations to avoid the loss of significance. Now we show different techniques.

Example

Consider the function $f(x) = e^x - 1$. In the neighborhood of $x = 0$ we again need to subtract two nearly equal numbers, but here we cannot use an algebraic identity to avoid it. But here we can consider the Taylor series of the exponential function, and we get

$$\begin{aligned} f(x) &= 1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} + \cdots - 1 \\ &= x + \frac{x^2}{2} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!} + \cdots . \end{aligned}$$

It is worth to take a finite approximation of this infinite series, and use it as an approximation of the function value $f(x)$.

Example

Evaluate the numerical value of

$$\frac{15^{40}}{40!}.$$

If we compute the numerator and the denominator separately first, then we run into the problem of overflowing the calculation. On the other hand, we know that

$$\lim_{n \rightarrow \infty} \frac{a^n}{n!} = 0,$$

so the result must be a small number. We rearrange the computation as follows:

$$\frac{15^{40}}{40!} = \frac{15}{40} \cdot \frac{15}{39} \cdot \frac{15}{38} \cdots \frac{15}{1}.$$

Here in each steps, the expressions we need to evaluate belong to the range which can be stored in the computer.

Example (cont.)

This formula can be computed with a simple **for** cycle:

```
 $y \leftarrow 15$   
for  $i = 2, \dots, 40$  do  
     $y \leftarrow y \cdot \frac{15}{i}$   
end do  
output( $y$ )
```

The result is 0.135521 (with single precision).

Example

Compute the sum

$$A = 1.000 + 0.0003 + 0.0003 + \cdots + 0.0003 = 1.000 + \sum_{i=1}^{1000} 0.0003$$

using a 4-digit arithmetic. We perform the additions from left to right, so first we compute

$$1.000 + 0.0003 = 1.0003 = 1.000$$

after the rounding. Adding the next number to it, because of the rounding to 4 digits, we get again

$$1.000 + 0.0003 + 0.0003 = 1.000.$$

Hence we get the numerical result

$$A = 1.000.$$

Example (cont.)

Consider the same sum, but in the opposite order:

$$B = 0.0003 + 0.0003 + \cdots + 0.0003 + 1.000 = \sum_{i=1}^{1000} 0.0003 + 1.000.$$

First we compute

$$0.0003 + 0.0003 = 0.0006.$$

Then we can continue:

$$0.0003 + 0.0003 + 0.0003 = 0.0009,$$

and similarly we get,

$$B = \sum_{i=1}^{1000} 0.0003 + 1.000 = 0.3 + 1.000 = 1.300.$$

The addition with floating point arithmetic is not commutative numerically.

Conclusion: in computing sums with several terms, it is advantageous to do the computation in an increasing order of the terms.